

به نام خدا

گزارش پروژه درس نظریه زبان ها و ماشین ها

Theory of Languages and Automata – Practical Project

اعضای گروه:

403541474 امیرحسین رضایی ورزقانی

403521024 امین اسلامیان

403411123 مهدی اصغری

استادان درس:

دکتر فرزانه غیور باغبانی

دکتر رضا انتظاری ملکی

تاریخ:

20 تیر 1405

نیمسال دوم 1404-1405

دانشگاه علم و صنعت ایران

مهندسی کامپیوتر



فهرست مطالب

مقدمه	3
چکیده	5
مبانی نظری	7
طراحی و پیاده‌سازی شبیه‌ساز ماشین تورینگ	10
نتیجه‌گیری	24

نظریه زبان‌ها و ماشین‌ها یکی از مهم‌ترین شاخه‌های علوم کامپیوتر است که مبانی ریاضی محاسبات، زبان‌های صوری و مدل‌های مختلف محاسباتی را مورد مطالعه قرار می‌دهد. بسیاری از مفاهیمی که امروزه در طراحی الگوریتم‌ها، زبان‌های برنامه‌نویسی، معماری رایانه‌ها و حتی هوش مصنوعی مورد استفاده قرار می‌گیرند، ریشه در این حوزه دارند. هدف اصلی این علم، پاسخ به پرسش‌هایی نظیر «چه مسائلی قابل حل هستند؟»، «چگونه می‌توان یک فرایند محاسباتی را مدل‌سازی کرد؟» و «حدود توانایی مدل‌های محاسباتی چیست؟» است.

ماشین تورینگ به عنوان شناخته‌شده‌ترین مدل محاسباتی، مبنای تعریف مفهوم محاسبه‌پذیری و تصمیم‌پذیری محسوب می‌شود و نقش مهمی در درک محدودیت‌های محاسبات ایفا می‌کند. در مقابل، اتوماتاهای سلولی نشان می‌دهند که چگونه مجموعه‌ای از قوانین بسیار ساده و محلی می‌توانند رفتارهایی پیچیده، پویا و حتی معادل با یک ماشین تورینگ ایجاد کنند. همین ویژگی باعث شده است که این مدل‌ها علاوه بر اهمیت نظری، در حوزه‌هایی مانند شبیه‌سازی سیستم‌های طبیعی، علوم پیچیدگی و محاسبات توزیع‌شده نیز کاربرد گسترده‌ای پیدا کنند.

در این پروژه تلاش شده است تا این مفاهیم صرفاً در سطح نظری باقی نمانند و از طریق پیاده‌سازی عملی مورد بررسی قرار گیرند. به همین منظور، ابتدا یک شبیه‌ساز کامل ماشین تورینگ توسعه داده شده که قابلیت اجرای ماشین‌های مختلف، مشاهده وضعیت‌های میانی، انجام محاسبات ریاضی و بررسی مسئله Busy Beaver را فراهم می‌کند. سپس دو اتوماتای سلولی مشهور شامل بازی زندگی کانوی و مورچه لنگتون پیاده‌سازی شده‌اند و در ادامه، با طراحی گیت‌های منطقی مبتنی بر گلایدرها، مفهوم جامعیت تورینگ در اتوماتاهای سلولی به صورت عملی نمایش داده شده است.

Introduction

Theory of Languages and Automata forms one of the fundamental pillars of computer science by providing the mathematical foundations of computation, algorithms, and computational systems. The Turing Machine serves as the canonical model of computation and establishes the basis for concepts such as computability, decidability, halting behavior, and computational universality. In contrast, cellular automata demonstrate how simple local rules can generate highly complex global behaviors capable of universal computation.

The primary objective of this project is to provide practical implementations of these theoretical concepts. A complete Turing Machine simulator is first developed together with unary arithmetic operations and the Busy Beaver problem. The project then implements Conway's Game of Life and Langton's Ant, followed by the construction of digital logic gates using glider interactions to experimentally demonstrate the Turing completeness of cellular automata.

چکیده

پروژه حاضر با هدف بررسی عملی مفاهیم بنیادی نظریه زبان‌ها و ماشین‌ها، دو دسته از مهم‌ترین مدل‌های محاسباتی یعنی ماشین‌های تورینگ و اتوماتاهای سلولی را در قالب یک مجموعه نرم‌افزاری جامع پیاده‌سازی می‌کند. در بخش نخست، یک شبیه‌ساز کامل ماشین تورینگ طراحی شده است که امکان اجرای مرحله‌به‌مرحله، مشاهده پیکربندی‌های ماشین، انجام محاسبات در نمایش یکانی و بررسی مسئله مشهور Busy Beaver را فراهم می‌کند. این بخش، مفاهیمی همچون محاسبه‌پذیری، توقف ماشین و نقش ماشین تورینگ به عنوان مدل مرجع محاسبات را به صورت عملی مورد بررسی قرار می‌دهد.

در بخش دوم، دو نمونه شناخته‌شده از اتوماتاهای سلولی شامل بازی زندگی کانوی (Conway's Game of Life) و مورچه لنگتون (Langton's Ant) پیاده‌سازی شده‌اند. علاوه بر شبیه‌سازی رفتار این سیستم‌ها، الگوهای مختلف آن‌ها مورد بررسی قرار گرفته و با استفاده از برخورد گلایدرها، گیت‌های منطقی پایه طراحی شده‌اند تا مفهوم جامعیت تورینگ و قابلیت انجام محاسبات عمومی در این مدل‌ها به صورت تجربی نمایش داده شود.

در مجموع، این پروژه پلی میان مباحث نظری و پیاده‌سازی عملی ایجاد کرده و نشان می‌دهد که چگونه مدل‌های محاسباتی متفاوت، با وجود تفاوت در ساختار و قواعد، از نظر توان محاسباتی معادل بوده و قادر به انجام هر محاسبه قابل الگوریتمی هستند.

Abstract

This project provides practical implementations of two fundamental computational models studied in the Theory of Languages and Automata: Turing Machines and Cellular Automata. The first part develops a complete Turing Machine simulator capable of step-by-step execution, configuration tracing, unary arithmetic computations, and exploration of the famous Busy Beaver problem. These implementations provide practical insight into computability, halting behavior, and the role of Turing Machines as the canonical model of computation.

The second part implements two well-known cellular automata, Conway's Game of Life and Langton's Ant. Beyond simulation, various cellular patterns are investigated, and digital logic gates are constructed through glider interactions to experimentally demonstrate Turing completeness and universal computation.

Overall, the project bridges theoretical concepts with practical implementation and illustrates that different computational models, despite their structural differences, possess equivalent computational power and are capable of performing any algorithmically computable task.

مبانی نظری

نظریه محاسبات به مطالعه مدل‌های انتزاعی محاسبه و توانایی آن‌ها در حل مسائل مختلف می‌پردازد. در این حوزه، هدف تنها طراحی الگوریتم نیست، بلکه بررسی حدود و مرزهای محاسبات، امکان یا عدم امکان حل یک مسئله و همچنین مقایسه توان محاسباتی مدل‌های مختلف است. ماشین‌های تورینگ، اتوماتاهای سلولی، زبان‌های صوری و سیستم‌های بازنویسی از جمله مهم‌ترین مدل‌هایی هستند که برای مطالعه این مفاهیم مورد استفاده قرار می‌گیرند.

ماشین تورینگ یکی از ساده‌ترین و در عین حال قدرتمندترین مدل‌های محاسباتی است. این ماشین از مجموعه‌ای متناهی از حالت‌ها، یک نوار حافظه، هد خواندن و نوشتن و تابع انتقال تشکیل شده است. در هر مرحله، هد ماشین نماد جاری را خوانده، بر اساس تابع انتقال وضعیت بعدی را انتخاب می‌کند، در صورت نیاز نماد جدیدی روی نوار می‌نویسد و سپس یک خانه به سمت چپ یا راست حرکت می‌کند. دنباله این انتقال‌ها، اجرای یک الگوریتم را تشکیل می‌دهد و در نهایت ماشین در یکی از حالت‌های پذیرش یا رد متوقف می‌شود.

یکی از مهم‌ترین مفاهیم مرتبط با ماشین تورینگ، مسئله توقف (Halting Problem) است که نشان می‌دهد هیچ الگوریتم عمومی برای تشخیص توقف یا عدم توقف تمام ماشین‌های تورینگ وجود ندارد. مسئله Busy Beaver نیز از همین مفهوم سرچشمه می‌گیرد. در این مسئله، هدف یافتن ماشینی است که با تعداد مشخصی حالت، پیش از توقف بیشترین تعداد نماد یا بیشترین تعداد گام ممکن را تولید کند. رشد بسیار سریع تابع Busy Beaver نشان‌دهنده محدودیت‌های بنیادی محاسبات و ارتباط مستقیم آن با مسئله توقف است.

اتوماتاهای سلولی دسته دیگری از مدل‌های محاسباتی هستند که از شبکه‌ای از سلول‌ها تشکیل می‌شوند. هر سلول دارای مجموعه محدودی از حالت‌ها بوده و وضعیت آن در هر گام زمانی تنها بر اساس وضعیت همسایگان خود تعیین می‌شود. اگرچه قوانین این سیستم‌ها بسیار ساده هستند، اما تعامل تعداد زیادی از سلول‌ها رفتارهایی بسیار پیچیده ایجاد می‌کند که نمونه‌ای از پدیده ظهور (Emergence) محسوب می‌شود.

بازی زندگی کانوی یکی از مشهورترین اتوماتاهای سلولی دوبعدی است که تنها با چهار قانون ساده تعریف می‌شود. با وجود سادگی این قوانین، الگوهای بسیار متنوعی مانند Oscillator، Glider و Spaceship در آن شکل می‌گیرند. پژوهش‌های انجام‌شده نشان داده‌اند که این سیستم از نظر توان محاسباتی معادل یک ماشین تورینگ بوده و می‌توان با طراحی مدارهای منطقی مناسب، هر الگوریتم قابل محاسبه را در آن پیاده‌سازی کرد.

مورچه لنگتون نمونه دیگری از اتوماتاهای سلولی است که رفتار آن توسط حرکت یک عامل روی شبکه تعیین می‌شود. این سیستم در ابتدا رفتاری شبه‌تصادفی از خود نشان می‌دهد، اما پس از طی تعداد زیادی گام، ساختاری منظم موسوم به بزرگراه (Highway) ایجاد می‌کند. این ویژگی نمونه‌ای از ظهور نظم از دل قوانین ساده محلی است و اهمیت اتوماتاهای سلولی را در مطالعه سیستم‌های پیچیده نشان می‌دهد.

یکی از مهم‌ترین مفاهیم مشترک میان ماشین‌های تورینگ و اتوماتاهای سلولی، جامعیت تورینگ (Turing Completeness) است. یک مدل محاسباتی زمانی جامع تورینگ نامیده می‌شود که بتواند هر تابع قابل محاسبه را، با صرف منابع کافی، محاسبه کند. در این پروژه، این مفهوم از دو دیدگاه بررسی می‌شود؛ ابتدا با پیاده‌سازی مستقیم ماشین تورینگ و سپس با ساخت گیت‌های منطقی AND و NOT در بازی زندگی کانوی که نشان می‌دهد حتی سیستم‌هایی با قوانین بسیار ساده نیز قادر به انجام محاسبات عمومی هستند.

در پیاده‌سازی نرم‌افزاری این پروژه، از زبان برنامه‌نویسی Python برای توسعه مدل‌ها استفاده شده است. شبیه‌ساز ماشین تورینگ با استفاده از Generatorها امکان اجرای مرحله‌به‌مرحله و مشاهده پیکربندی ماشین را فراهم می‌کند، در حالی که در بخش اتوماتاهای سلولی از ساختارهای داده مناسب و کتابخانه‌های گرافیکی برای نمایش روند تکامل سیستم استفاده شده است. این رویکرد موجب می‌شود مفاهیم نظری درس در قالب یک محیط عملی، قابل مشاهده، آزمایش و تحلیل باشند.

طراحی و پیاده‌سازی شبیه‌ساز ماشین تورینگ

پس از بررسی مبانی نظری ماشین تورینگ، در این بخش به پیاده‌سازی عملی این مدل محاسباتی پرداخته می‌شود. هدف اصلی، توسعه یک شبیه‌ساز عمومی است که بتواند تنها با دریافت مجموعه‌ای از قوانین انتقال، رفتار هر ماشین تورینگ دلخواه را شبیه‌سازی کند. به همین منظور، ساختار برنامه به گونه‌ای طراحی شده است که اجزای مختلف ماشین شامل حالت‌ها، نوار، هد خواندن و نوشتن و تابع انتقال به صورت مستقل مدیریت شوند و امکان استفاده مجدد از شبیه‌ساز برای مسائل مختلف فراهم گردد.

در این پیاده‌سازی، کلاس TuringMachine هسته اصلی برنامه را تشکیل می‌دهد و تمامی عملیات مربوط به اجرای ماشین در این کلاس انجام می‌شود. علاوه بر اجرای ماشین، امکاناتی نظیر بررسی پذیرش یا رد ورودی، نمایش مرحله به مرحله فرآیند اجرا، محدودسازی تعداد گام‌های شبیه‌سازی و ثبت هشدارهای لازم نیز در همین کلاس پیاده‌سازی شده‌اند. این طراحی باعث شده است که شبیه‌ساز علاوه بر انجام محاسبات، به عنوان ابزاری مناسب برای آزمایش، تحلیل و اشکال‌زدایی ماشین‌های تورینگ نیز مورد استفاده قرار گیرد.

در ادامه، اجزای مختلف این کلاس و نحوه پیاده‌سازی هر یک از بخش‌های اصلی آن به تفصیل بررسی می‌شود.

زیربخش اول: طراحی کلاس TuringMachine

اولین گام در پیاده‌سازی شبیه‌ساز، طراحی کلاسی بود که بتواند تمامی اجزای ماشین تورینگ را در قالب یک شیء واحد مدیریت کند. این کلاس مسئول نگهداری اطلاعات پایه ماشین شامل مجموعه قوانین انتقال، حالت شروع، حالت پذیرش، حالت رد و نماد خانه خالی نوار است. همچنین تمامی عملیات مربوط به اجرای ماشین و تعامل با آن از طریق متدهای همین کلاس انجام می‌شود. به این ترتیب، برای تعریف هر ماشین تورینگ جدید تنها کافی است قوانین انتقال آن در زمان ایجاد شیء به کلاس داده شوند و سایر قابلیت‌های شبیه‌ساز بدون نیاز به تغییر مجدد قابل استفاده خواهند بود.

کد:

```
class TuringMachine:
    """Turing machine simulator class."""

    def __init__(
        self,
        transitions,
        start_state='q0',
        accept_state='qa',
        reject_state='qr',
        blank_symbol=''
    ):
        self.transitions = transitions
        self.start_state = start_state
        self.accept_state = accept_state
        self.reject_state = reject_state
        self.blank_symbol = blank_symbol
```

در این بخش، سازنده (Constructor) کلاس وظیفه مقداردهی اولیه اجزای اصلی ماشین تورینگ را بر عهده دارد. مهم‌ترین ورودی این متد، متغیر transitions است که تمامی قوانین انتقال ماشین را در خود نگهداری می‌کند و منطق عملکرد ماشین بر اساس آن تعیین می‌شود. علاوه بر آن، چهار پارامتر دیگر نیز برای افزایش انعطاف‌پذیری کلاس در نظر گرفته شده‌اند که به ترتیب حالت شروع (start_state)، حالت پذیرش (accept_state)، حالت رد (reject_state) و نماد خانه خالی نوار (blank_symbol) را مشخص می‌کنند.

تمامی این پارامترها در متغیرهای عضو کلاس ذخیره می‌شوند تا در طول اجرای ماشین توسط سایر متدها مورد استفاده قرار گیرند. مقدارهای پیش‌فرض در نظر گرفته شده (q0) برای شروع، qa برای پذیرش و qr برای رد (باعث می‌شود در اکثر ماشین‌های تورینگ نیازی به تعیین مجدد این مقادیر نباشد و تنها مجموعه قوانین انتقال به سازنده ارسال شود. با این حال، در صورت نیاز می‌توان این مقادیر را هنگام ایجاد شیء تغییر داد و ماشین‌هایی با نام‌گذاری متفاوت را نیز بدون هیچ تغییری در سایر بخش‌های برنامه اجرا کرد.

به طور کلی، طراحی این کلاس باعث شده است که شبیه‌ساز به صورت کاملاً عمومی (Generic) عمل کند؛ به گونه‌ای که منطق اجرای ماشین مستقل از مسئله مورد نظر باشد و تنها با تغییر مجموعه قوانین انتقال، بتوان ماشین‌های مختلفی مانند ماشین‌های تشخیص زبان، ماشین جمع، ماشین ضرب و Busy Beaver را با استفاده از همین کلاس پیاده‌سازی و اجرا کرد.

زیر بخش دوم: ساختار داده‌ی قوانین انتقال (Transition Function)

اساسی‌ترین بخش هر ماشین تورینگ، تابع انتقال آن است؛ زیرا رفتار ماشین در هر مرحله تنها بر اساس این تابع تعیین می‌شود. در واقع، تابع انتقال مشخص می‌کند که اگر ماشین در یک حالت مشخص قرار داشته باشد و نماد خاصی را از روی نوار بخواند، باید به کدام حالت منتقل شود، چه نمادی را روی نوار بنویسد و هد را در چه جهتی حرکت دهد. از آنجا که تمام منطق اجرای ماشین در این قوانین نهفته است، انتخاب یک ساختار داده مناسب برای ذخیره‌سازی آن‌ها اهمیت زیادی دارد.

در این پروژه، قوانین انتقال به صورت یک دیکشنری (Dictionary) در زبان Python پیاده‌سازی شده‌اند. این انتخاب باعث می‌شود عملیات جستجوی قانون متناظر با هر وضعیت در زمان ثابت ($O(1)$) انجام شود و شبیه‌سازی ماشین حتی در تعداد زیاد انتقال‌ها نیز با سرعت مناسبی اجرا گردد. علاوه بر این، استفاده از دیکشنری خوانایی کد را افزایش داده و تعریف ماشین‌های جدید را تنها به اضافه کردن مجموعه‌ای از قوانین انتقال محدود می‌کند.

کد

```
transitions = {  
    ('q0', '#'): ('saw_#', '#', 'R'),  
    ('saw_#', ''): ('qa', '', 'R'),  
}
```

توضیح کد

در این ساختار، کلید (Key) هر عضو دیکشنری یک زوج مرتب شامل حالت فعلی و نماد خوانده شده از روی نوار است. مقدار (Value) متناظر نیز یک سه تایی شامل حالت بعدی، نمادی که باید روی نوار نوشته شود و جهت حرکت هد است. بنابراین هر قانون انتقال دقیقاً متناظر با تعریف ریاضی تابع انتقال ماشین تورینگ بوده و نگاشت مستقیمی میان مدل نظری و پیاده سازی نرم افزاری ایجاد می کند.

برای مثال، قانون

('q0', '#'): ('saw_#', '#', 'R')

بیان می کند که اگر ماشین در حالت q0 قرار داشته باشد و نماد # را بخواند، ابتدا همان نماد # را روی نوار می نویسد، سپس وارد حالت saw_# شده و در نهایت هد را یک خانه به سمت راست حرکت می دهد. سایر قوانین نیز دقیقاً با همین ساختار تعریف می شوند و مجموعه آن ها رفتار کامل ماشین را مشخص می کند.

یکی از مزایای این طراحی، استقلال کامل شبیه ساز از نوع مسئله است. کلاس TuringMachine هیچ اطلاعی از زبان ورودی یا هدف ماشین ندارد و تنها با استفاده از همین دیکشنری، مراحل اجرا را مدیریت می کند. به همین دلیل، برای ساخت ماشین های مختلف مانند ماشین تشخیص زبان، ماشین جمع، ماشین ضرب یا Busy Beaver، تنها کافی است مجموعه قوانین انتقال تغییر داده شود و هیچ تغییری در منطق اصلی شبیه ساز لازم نخواهد بود.

زیربخش سوم: طراحی نوار (Tape)

نوار ماشین تورینگ نقش حافظه اصلی سیستم را ایفا می‌کند و تمام عملیات خواندن، نوشتن و حرکت هد روی آن انجام می‌شود. در پیاده‌سازی حاضر، به جای استفاده از یک آرایه بزرگ یا ساختارهای پیچیده، نوار به سه بخش مجزا تقسیم شده است: بخش سمت چپ هد، نماد فعلی زیر هد و بخش سمت راست هد. این نمایش علاوه بر سادگی، امکان مدیریت آسان حرکت هد و تغییر محتوای نوار را نیز فراهم می‌کند.

برای نمایش وضعیت جاری ماشین، از یک دیکشنری با عنوان configuration استفاده شده است که علاوه بر وضعیت ماشین، اطلاعات مربوط به نوار را نیز نگهداری می‌کند. این ساختار، هر لحظه یک تصویر کامل از وضعیت اجرای ماشین در اختیار برنامه قرار می‌دهد و در ادامه توسط متد `debug()` برای نمایش مرحله به مرحله اجرای ماشین مورد استفاده قرار می‌گیرد.

کد

```
current_symbol = (
    input_[0]
    if len(input_) > 0
    else self.blank_symbol
)
configuration = {
    "state": self.start_state,
    "left_hand_side": [],
    "symbol": current_symbol,
    "right_hand_side": list(input_[1:])
}
```

توضیح کد

در ابتدای اجرای ماشین، ورودی روی نوار قرار می‌گیرد و هد روی اولین نماد آن قرار می‌گیرد. اگر ورودی تهی باشد، هد مستقیماً روی نماد Blank قرار خواهد گرفت. به همین دلیل، ابتدا مقدار `current_symbol` تعیین می‌شود تا مشخص گردد هد در ابتدای اجرا چه نمادی را مشاهده می‌کند.

سپس ساختار `configuration` ایجاد می‌شود. کلید `state` وضعیت فعلی ماشین را نگهداری می‌کند. کلید `left_hand_side` شامل تمامی نمادهای سمت چپ هد است که نزدیک‌ترین خانه به هد در انتهای لیست قرار می‌گیرد. کلید `symbol` نماد فعلی زیر هد را نگهداری می‌کند و `right_hand_side` نیز شامل تمام نمادهای سمت راست هد است. این نمایش باعث می‌شود عملیات حرکت هد تنها با جابه‌جایی عناصر بین این سه بخش انجام شود و نیازی به انتقال کل نوار یا تغییر اندیس‌های آن نباشد.

حرکت هد روی نوار

پس از اعمال هر قانون انتقال، هد باید مطابق جهت تعیین‌شده یک خانه به سمت چپ یا راست حرکت کند. در حرکت به سمت راست، ابتدا نماد فعلی به انتهای بخش چپ نوار منتقل شده و سپس اولین نماد بخش راست به عنوان نماد جدید زیر هد انتخاب می‌شود. اگر بخش راست خالی باشد، هد وارد خانه‌ای شامل نماد Blank خواهد شد.

در حرکت به سمت چپ نیز روند معکوس انجام می‌شود؛ بدین صورت که نماد فعلی در ابتدای بخش راست قرار گرفته و آخرین نماد موجود در بخش چپ به زیر هد منتقل می‌شود. در صورتی که هد از ابتدای نوار عبور کند، در این نسخه از پروژه نوار به صورت تک‌طرفه نامتناهی (Singly Infinite Tape) در نظر گرفته شده است؛ بنابراین با ثبت یک پیام هشدار توسط ماژول logging، هد روی یک خانه Blank قرار می‌گیرد. این رفتار مطابق صورت اولیه پروژه بوده و در بخش مربوط به Busy Beaver، ساختار نوار برای پشتیبانی از نوار نامتناهی دوطرفه توسعه داده خواهد شد.

نماد Blank

نماد خانه خالی نوار در این پروژه توسط متغیر `blank_symbol` مشخص شده است که مقدار پیش‌فرض آن رشته تهی (") است. هر زمان هد وارد ناحیه‌ای شود که قبلاً مقداری در آن نوشته نشده باشد، این نماد به عنوان محتوای آن خانه در نظر گرفته می‌شود. استفاده از یک متغیر مستقل برای نماد Blank باعث شده است که در صورت نیاز بتوان بدون تغییر سایر بخش‌های برنامه، نماد دیگری را به عنوان خانه خالی تعریف کرد.

زیربخش چهارم: الگوریتم اجرای ماشین

مهم‌ترین بخش شبیه‌ساز ماشین تورینگ، متد `run()` است که وظیفه اجرای مرحله به مرحله ماشین را بر عهده دارد. این متد بر اساس قوانین انتقال تعریف شده، وضعیت جاری ماشین را به روزرسانی کرده و تا رسیدن به یکی از حالت‌های پذیرش یا رد، اجرای ماشین را ادامه می‌دهد. علاوه بر این، به دلیل پیاده‌سازی این متد به صورت `Generator`، امکان مشاهده تمامی پیکربندی‌های میانی ماشین در طول اجرا نیز فراهم شده است. این ویژگی علاوه بر اشکال زدایی، برای بررسی روند اجرای ماشین و تحلیل عملکرد آن نیز بسیار کاربردی است.

الگوریتم اجرا به گونه‌ای طراحی شده است که در هر مرحله ابتدا وضعیت فعلی ماشین بررسی می‌شود. در صورتی که ماشین به حالت پذیرش یا رد رسیده باشد، اجرای آن خاتمه می‌یابد. در غیر این صورت، قانون انتقال متناظر با وضعیت فعلی از دیکشنری قوانین استخراج شده، عملیات نوشتن روی نوار و حرکت هد انجام می‌شود و در نهایت پیکربندی جدید ماشین تولید خواهد شد. این روند تا زمان توقف ماشین به صورت تکرار شونده ادامه پیدا می‌کند.

کد

```
def run(self, input_):
```

توجه: به دلیل طولانی بودن متد run()، در متن گزارش، کل بدنه این متد به صورت کامل آورده شده است.

توضیح کد

در ابتدای متد، ورودی روی نوار قرار گرفته و اولین پیکربندی ماشین ایجاد می‌شود. اگر رشته ورودی تهی باشد، هد مستقیماً روی نماد Blank قرار خواهد گرفت. پس از مقداردهی اولیه، اولین وضعیت ماشین با استفاده از دستور yield تولید می‌شود تا کاربر بتواند وضعیت اولیه ماشین را نیز مشاهده کند.

سپس ماشین وارد یک حلقه نامحدود می‌شود که تا زمان توقف ادامه خواهد داشت. در ابتدای هر تکرار، ابتدا بررسی می‌شود که آیا ماشین به حالت پذیرش (accept_state) یا رد (reject_state) رسیده است یا خیر. در صورت رسیدن به هر یک از این دو حالت، نتیجه مناسب (Accept) یا (Reject) به همراه آخرین پیکربندی تولید شده و اجرای متد خاتمه پیدا می‌کند.

اگر وضعیت فعلی در میان قوانین انتقال تعریف نشده باشد، به این معناست که ماشین قادر به ادامه اجرای خود نیست؛ بنابراین ورودی رد شده و متد پایان می‌یابد. در غیر این صورت، قانون انتقال متناظر استخراج شده و سه مؤلفه آن شامل حالت بعدی، نماد جدید و جهت حرکت هد از یکدیگر جدا می‌شوند. سپس ابتدا نماد جدید روی نوار نوشته شده و وضعیت ماشین به حالت جدید تغییر می‌کند.

در ادامه، با توجه به جهت حرکت (L یا R)، ساختار نوار به‌روزرسانی می‌شود. در حرکت به سمت راست، نماد فعلی به بخش چپ نوار منتقل شده و اولین نماد سمت راست به زیر هد می‌آید. در حرکت به سمت چپ نیز عملیات معکوس انجام می‌شود. در صورتی که هد از ابتدای نوار عبور کند، پیام هشدار مناسبی توسط ماژول logging ثبت شده و هد روی یک خانه Blank قرار می‌گیرد. همچنین اگر جهت حرکت مقداری غیر از L یا R داشته باشد، یک استثناء از نوع ValueError ایجاد می‌شود تا از اجرای قوانین نامعتبر جلوگیری گردد.

در پایان هر انتقال، نسخه‌ای مستقل از پیکربندی فعلی با استفاده از `copy.deepcopy()` تولید شده و توسط دستور `yield` در اختیار فراخواننده قرار می‌گیرد. استفاده از `deepcopy` باعث می‌شود تغییرات مراحل بعدی، وضعیت پیکربندی‌های قبلی را تحت تأثیر قرار ندهد و تمامی مراحل اجرا به صورت مستقل قابل مشاهده و بررسی باشند.

زیربخش پنجم: متدهای کمکی

علاوه بر متد اصلی `run()`، برای ساده‌تر شدن استفاده از شبیه‌ساز، سه متد کمکی نیز در کلاس `TuringMachine` پیاده‌سازی شده‌اند. این متدها وظیفه بررسی نتیجه اجرای ماشین و نمایش روند اجرای آن را بر عهده دارند و باعث می‌شوند کاربر بدون درگیر شدن با جزئیات `Generator`، بتواند به سادگی ماشین را اجرا و نتیجه آن را مشاهده کند.

متد `accepts()`

کد

```
def accepts(self, input_, step_limit=100):
```

```
...
```

توضیح کد

این متد برای بررسی پذیرش یا رد یک ورودی طراحی شده است. در ابتدا با فراخوانی متد `run()`، `Generator` مربوط به اجرای ماشین ایجاد می‌شود. سپس با استفاده از تابع `islice` حداکثر تعداد مشخصی از مراحل اجرا (بر اساس پارامتر `step_limit`) بررسی می‌شوند. در صورتی که ماشین در این بازه زمانی به حالت پذیرش برسد، مقدار `True` و در صورت رسیدن به حالت رد، مقدار `False` بازگردانده می‌شود.

در صورتی که ماشین تا رسیدن به محدودیت تعیین‌شده متوقف نشود، پیام هشدار مناسبی ثبت شده و مقدار `None` بازگردانده می‌شود. این قابلیت از گرفتار شدن برنامه در حلقه‌های بی‌نهایت جلوگیری کرده و امکان آزمایش ماشین‌هایی را که احتمال توقف ندارند نیز فراهم می‌کند.

متد rejects()

کد

```
def rejects(self, input_, **kwargs):  
    ...
```

توضیح کد

این متد در واقع مکمل متد `accepts()` و برای افزایش خوانایی برنامه پیاده‌سازی شده است. ابتدا نتیجه متد `accepts()` دریافت می‌شود؛ اگر مقدار بازگشتی `None` باشد، همان مقدار بدون تغییر بازگردانده می‌شود. در غیر این صورت، نتیجه منطقی معکوس شده و به عنوان پاسخ متد بازگردانده می‌شود. به این ترتیب، بدون اجرای مجدد ماشین می‌توان نتیجه رد شدن ورودی را نیز به سادگی بررسی کرد.

متد debug()

کد

```
def debug(self, input_, step_limit=100, colored=False):  
    ...
```

توضیح کد

هدف از این متد، نمایش مرحله به مرحله اجرای ماشین برای اهداف آموزشی و اشکال‌زدایی است. این متد نیز همانند `accepts()` Generator تولید شده توسط متد `run()` استفاده می‌کند، اما به جای بررسی نتیجه نهایی، تمامی پیکربندی‌های تولید شده را به ترتیب نمایش می‌دهد.

برای نمایش صحیح محتوای نوار، ابتدا بخش چپ نوار که به صورت معکوس در لیست ذخیره شده است، با استفاده از تابع `reversed()` به ترتیب اصلی بازگردانده می‌شود. سپس نماد زیر هد درون کروش قرار گرفته و بخش راست نوار نیز به آن اضافه می‌شود. خروجی نهایی به صورت

```
state: left[symbol]right
```

نمایش داده می‌شود که در آن محل قرارگیری هد به وضوح مشخص است. این شیوه نمایش، روند تغییرات نوار و جابه‌جایی هد را در هر مرحله از اجرای ماشین به شکل ساده و قابل فهم نمایش می‌دهد و ابزار مناسبی برای تحلیل عملکرد قوانین انتقال محسوب می‌شود.

